Gebeya Week 1 Challenge

Full Technical Documentation

Slack Messages Analysis Project

Biniam Tsige | Batch 6 | 2026
Data Engineering & Machine Learning Engineering Track

Table of Contents

1. Project Overview

The Gebeya Week 1 Challenge is a comprehensive technical assessment covering Data Engineering and Machine Learning Engineering. The project analyses anonymized Slack messages from Gebeya Batch 6 — real communication data from a previous cohort of trainees. The challenge spans five tasks covering everything from basic Python environment setup to cloud deployment.

What is Slack?

Slack is a team messaging platform — similar to WhatsApp but designed for professional use. One of its key advantages is that all messages can be exported as structured JSON files. This makes it ideal for data analysis. Each channel (like a group chat) produces one JSON file per day, containing every message sent that day along with metadata like timestamps, reactions, and thread replies.

The Five Tasks

Task	Topic	Key Technologies
Task 1	Git, GitHub, EDA & Statistics	Python, pandas, matplotlib, Jupyter
Task 2	Data Science Components & ML	scikit-learn, TextBlob, MLFlow, NLTK
Task 3	Databases	MongoDB, PostgreSQL, pymongo, psycopg2
Task 4	Dashboard	Streamlit
Task 5	Deployment	Docker, AWS, Terraform, GitHub Actions

■ Data Privacy: The anonymized Slack data must NEVER be pushed to GitHub. The anonymized/ folder is added to .gitignore to prevent accidental uploads. This complies with Gebeya's data privacy and confidentiality requirements.

2. Environment Setup

What is Homebrew?

Homebrew is a package manager for Mac — think of it as the App Store for developer tools that run in the terminal. Instead of going to a website, downloading an installer, and clicking through setup screens, you simply type one command and Homebrew handles everything.

```
brew install mongodb-community # installs MongoDB brew install terraform # installs
Terraform brew install gh # installs GitHub CLI
```

Homebrew installs everything into **/opt/homebrew/** on Apple Silicon Macs. You can list everything installed via Homebrew with:

```
brew list # list all installed tools brew list --versions # list with version numbers
brew info postgresql # info about one specific tool
```

What is pip?

pip is Python's package manager — it installs Python libraries (code that runs inside Python). While Homebrew installs programs on your Mac, pip installs libraries that your Python code imports.

```
pip install pandas # installs pandas pip install scikit-learn # installs scikit-learn
pip list # list all installed packages pip show pandas # info about one package
```

Homebrew vs pip

	Homebrew	pip
What it installs	Programs & system tools	Python libraries
Where it goes	/opt/homebrew/	Inside venv/
Who uses it	Your whole Mac	Only Python code
Example	MongoDB, PostgreSQL, Docker	pandas, mlflow, nltk
List command	brew list	pip list

What is a Virtual Environment (venv)?

A virtual environment is an isolated Python workspace — a folder that contains its own Python interpreter and packages, completely separate from your system Python. This solves the problem of different projects needing different versions of the same package.

Real life analogy: Imagine you work on two different projects. Project A needs pandas version 1.0 and Project B needs pandas version 2.0. Without venvs, they would conflict. With venvs, each project has its own isolated environment — like two separate toolboxes.

```
# Create the virtual environment python3 -m venv gebvenv # Activate it (you must do this
every new terminal session) source gebvenv/bin/activate # You know it's active when you
```

```
see (gebvenv) in your prompt: # (gebvenv) btsm@Biniams-MacBook-Pro Gebeya-Tech % #
Install packages inside the venv pip install pandas matplotlib seaborn jupyter
```

The venv folder (**gebvenv/**) is disposable. If it breaks, delete it and recreate it. Your notebooks and source code live outside it and are safe.

Registering venv as a Jupyter Kernel

When Jupyter opens a notebook, it uses a "kernel" — the Python engine that runs the code. By default Jupyter uses the system Python, which does not have your venv packages. You must register your venv as a kernel so notebooks can use it.

```
source gebvenv/bin/activate pip install ipykernel python -m ipykernel install --user
--name=gebvenv --display-name="Python (gebvenv)"
```

Then in Jupyter: **Kernel** → **Change Kernel** → **Python (gebvenv)**

3. Project Structure

```
Gebeya-Tech/ ■■■ .github/workflows/ ← GitHub Actions CI/CD workflows ■ ■■■
flake8_check.yml ← checks code style on every push ■ ■■■ unittests.yml ← runs pytest
on every push ■ ■■■ docstring_tests.yml ← runs docstring tests on every push ■■■
.vscode/settings.json ← VSCode Flake8 configuration ■■■ app/ ■ ■■■ dashboard.py ←
Streamlit dashboard (Task 4) ■■■ notebooks/ ■ ■■■ parse_slack_data.ipynb ← Task 1 EDA
notebook ■ ■■■ task2_analysis.ipynb ← Task 2 ML notebook ■ ■■■ task3_databases.ipynb
← Task 3 databases notebook ■■■ src/ ■ ■■■ __init__.py ← marks src as a Python
package ■ ■■■ config.py ← stores path to data folder ■ ■■■ loader.py ←
SlackDataLoader class ■ ■■■ utils.py ← helper functions ■■■ tests/ ■ ■■■ __init__.py
■ ■■■ test_loader.py ← 5 unit tests for SlackDataLoader ■■■ .flake8 ← code style
configuration ■■■ .gitignore ← prevents data from going to GitHub ■■■ gebvenv/ ←
virtual environment (not pushed to GitHub) ■■■ Makefile ← shortcuts: make test, make
lint ■■■ requirements.txt ← list of Python packages ■■■ README.md ← project
description
```

Key File Explanations

src/config.py — stores the path to the anonymized data folder so every file knows where to find it without hardcoding a path.

```
import os DATA_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)),
"anonymized")
```

src/loader.py — the SlackDataLoader class with 3 methods:

- get_channels() — lists all 39 channel folders
- load_channel(channel) — reads all JSON files from one channel into a DataFrame
- load_all_channels() — combines all channels into one DataFrame (20,011 messages)

src/utils.py — helper functions used across all notebooks:

- get_top_users() — ranks users from most to least by a metric
- get_bottom_users() — ranks users from least to most by a metric
- parse_timestamp() — converts Slack's Unix timestamp (1661774400.0) to a real datetime
- add_time_columns() — adds datetime, date, and hour columns to any DataFrame

> ■ Why utils.py? These functions are used in multiple notebooks. Instead of copying the same code into every notebook, we write it once and import it wherever needed. If we fix a bug in utils.py, all notebooks automatically get the fix.

What is __pycache__?

When Python runs a .py file, it compiles it into faster bytecode and saves it in __pycache__/ as .pyc files. The next time you run the same file, Python loads the faster bytecode version. You never create or edit this folder — Python manages it automatically. It is excluded from GitHub via .gitignore.

Does this make Python a compiled language? Not quite. Python compiles to bytecode automatically behind the scenes, but that bytecode still needs the Python interpreter to run. A fully compiled language (like C) runs directly on the CPU with no interpreter needed. Python is called "compiled to bytecode, then interpreted."

4. Task 1 — Git, GitHub & EDA

What is Git?

Git is a version control system — it tracks every change you make to your code over time. Think of it like "track changes" in Microsoft Word, but for entire projects. You can go back to any previous version of your code at any time.

What is GitHub?

GitHub is a website that stores your Git repository online. Git lives on your local machine; GitHub is the cloud backup and collaboration platform. Other developers can see your code, suggest changes, and contribute.

What is Flake8?

Flake8 is a code style checker. It reads your Python code and flags anything that violates PEP 8 — Python's official style guide. Bad code (to Flake8) means: lines too long, missing spaces around operators, unused imports, inconsistent indentation etc. It does not check whether your code works — only whether it is formatted correctly.

What is CI/CD?

CI/CD stands for Continuous Integration / Continuous Deployment. It is the practice of automatically running checks every time you push code to GitHub. In this project, three GitHub Actions workflows run automatically on every push:

- flake8_check.yml — checks code style
- unittests.yml — runs all pytest tests
- docstring_tests.yml — runs examples in docstrings as tests

EDA Analysis — Questions Answered

Top & Bottom 10 Users — We analysed users by 4 metrics: message count (how many messages they sent), reply count (how many replies they received), reaction count (how many emoji reactions their messages got), and mention count (how many times others @mentioned them).

Top 10 Messages — Found the most replied-to, most reacted-to, and most mentioned messages.

Channel Activity Scatter Plot — Plotted all 39 channels on a 2D chart where X = number of messages, Y = replies + reactions. The channel in the top-right corner is the most active.

Reply Time Analysis — Calculated what fraction of messages receive a reply within 5 minutes. Plotted a scatter plot with X = time difference, Y = time of day, colour = channel.

5. Task 2 — Data Science Components & ML

Unit Tests

A unit test is a small piece of code that checks whether another piece of code works correctly. We wrote 5 tests for the SlackDataLoader class:

- test_load_channel_returns_dataframe — confirms the return type is a DataFrame
- test_load_channel_has_expected_columns — confirms columns: type, user, text, ts, channel
- test_load_all_channels_returns_dataframe — same for all-channel loading
- test_load_all_channels_has_expected_columns — same column check for all channels
- test_channel_column_matches_channel_name — every row has the correct channel label

```
python -m pytest tests/test_loader.py -v # Result: 5 passed in 27.96s
```

Time Difference Histograms

We analysed the distribution of time gaps between consecutive events. A histogram is a bar chart that groups data into buckets (e.g. "how many gaps were 0–5 minutes?"). We computed 4 distributions: consecutive messages, consecutive replies, consecutive reactions, and all events combined. Each chart shows a red median line.

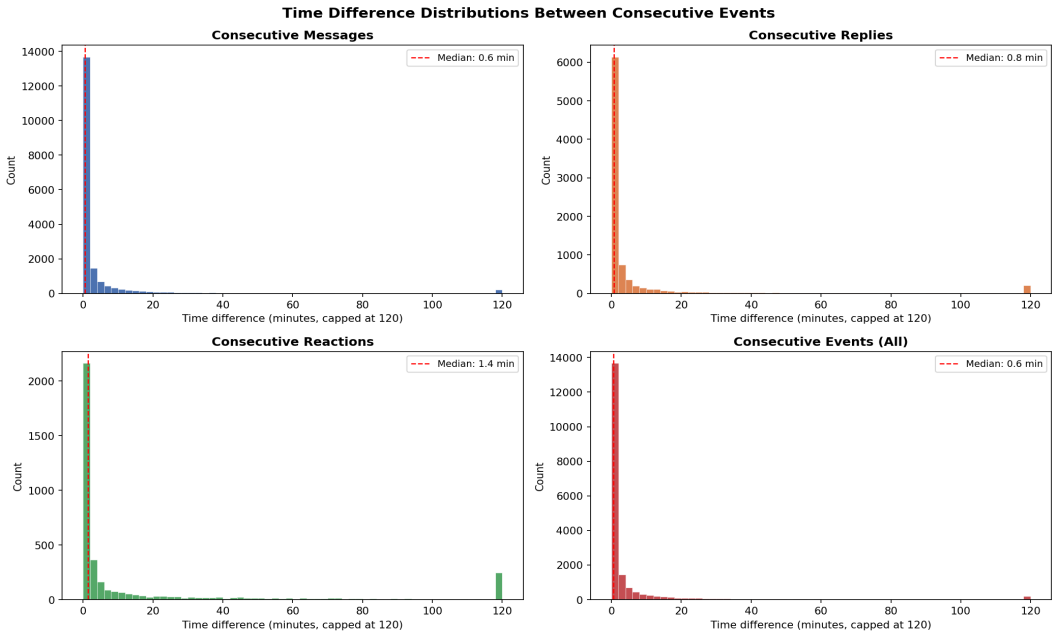

Figure 1: Time difference distributions between consecutive events

Message Classification

Every message was classified into one of 6 categories using rule-based heuristics (since no labeled training data exists): Question-Technical, Question-NonTechnical, Answer, Comment-Technical, Comment-NonTechnical, Other. Technical keywords included: error, code, python, git, model, docker, sql,

pandas, etc.

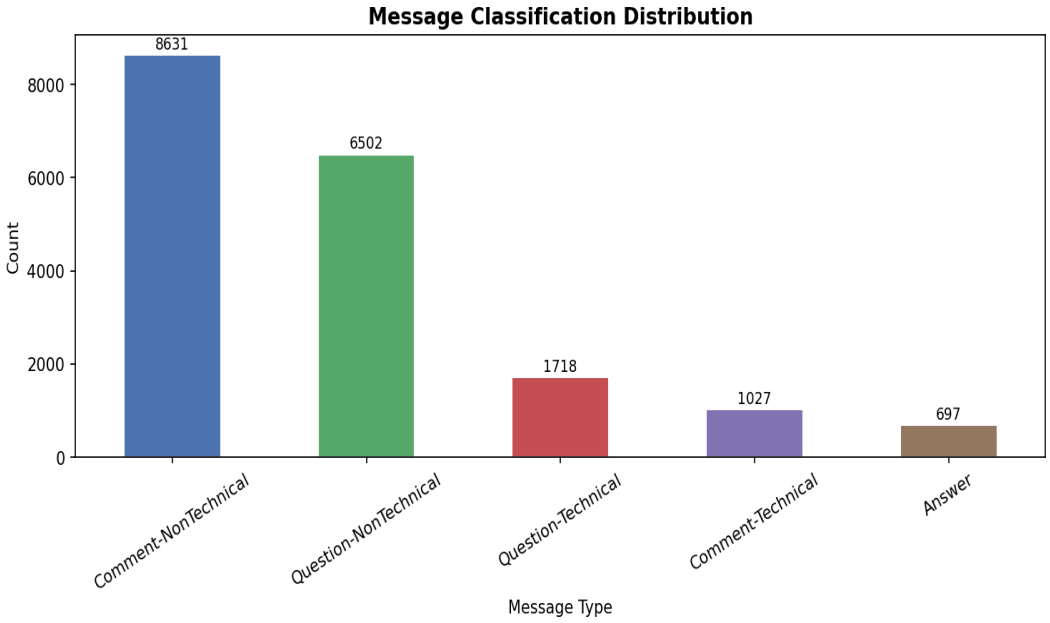

Figure 2: Distribution of message types across all channels

Topic Modelling (LDA)

Topic modelling automatically discovers hidden themes in thousands of documents without being told what the themes are. We used LDA — Latent Dirichlet Allocation. The idea: every message is a mixture of topics, every topic is a mixture of words. LDA works backwards from the words to figure out the topics. We found 10 topics.

Steps: clean text (remove Slack mentions, URLs, stop words), build a word-count matrix of top 2,000 words using CountVectorizer, train LDA with 10 topics, visualise results.

Why do topics look messy? LDA works best on long formal documents like news articles. Slack messages are short, informal, and full of slang — so topic boundaries are less clean. This is expected and should be mentioned in your report.

Figure 3: Top 10 LDA topics extracted from 20,000 Slack messages

Sentiment Analysis Over Time

Sentiment analysis measures the emotional tone of text — is it positive, negative, or neutral? TextBlob gives every piece of text a polarity score from -1 (very negative) to +1 (very positive). It works by looking up words in a pre-built dictionary where every word has a sentiment score.

We grouped all messages by day since training start, merged them into one big text per day, ran TextBlob on each day's text, and plotted the results. Green bars = positive days, red bars = negative days, blue line = smoothed trend using Savitzky-Golay filter.

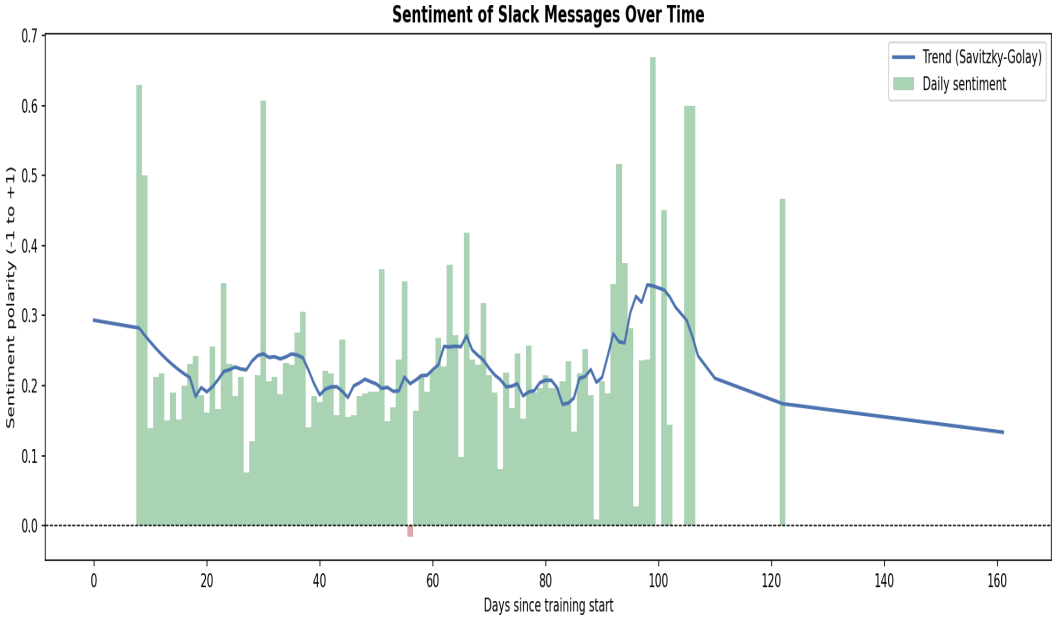

Figure 4: Sentiment polarity of Slack messages over time

MLFlow — Model Versioning

MLFlow tracks your machine learning experiments — like a lab notebook for ML. Every time you train a model, MLFlow records parameters, metrics, the model itself, and any artefacts (files). This is critical in real ML work where you train dozens of models with different settings and need to track which gave the best result.

Logged Item	Value
n_topics	10
max_iter	20
vocab_size	2,000
corpus_size	~18,000 documents
LDA perplexity	Computed after training
Mean sentiment	Computed from daily data
Model artefact	Trained LDA model (sklearn format)
Vectorizer artefact	CountVectorizer saved as .pkl

MLOps Components Summary

Component	What it does	Tool
Experiment Tracking	Logs params, metrics, artefacts per run	MLFlow
Feature Store	Central storage for ML features	PostgreSQL (Task 3)
Model Versioning	Tags models with version and stage	MLFlow Model Registry
Model Monitoring	Detects when deployed models degrade	Alibi-watcher / custom
CI/CD Pipeline	Auto-tests code on every push	GitHub Actions
Containerisation	Packages app + dependencies	Docker (Task 5)

6. Task 3 — MongoDB & PostgreSQL

Why Two Databases?

MongoDB and PostgreSQL solve completely different problems. Using only one would be the wrong tool for the job.

	MongoDB (NoSQL)	PostgreSQL (SQL)
Data structure	Documents (JSON-like)	Tables with fixed columns
Schema	Flexible — each document can differ	Rigid — all rows same structure
Best for	Raw, messy, unpredictable data	Clean, structured, computed data
Our use	Raw Slack JSON messages	Computed ML features
Analogy	Filing cabinet (any folder shape)	Spreadsheet (strict columns)

Installing MongoDB

```
# Add the MongoDB source to Homebrew brew tap mongodb/brew # Install MongoDB brew
install mongodb-community # Start the MongoDB server brew services start
mongodb-community # Verify it is running brew services list
```

MongoDB runs on **localhost:27017** — localhost means "this same computer", 27017 is the port (door number) MongoDB listens on.

MongoDB Schema Design — 3 Collections

A collection in MongoDB is like a table in SQL — a group of related documents. We split the Slack data into 3 collections for query speed and streaming efficiency:

- **messages** — 10,814 top-level messages
- **replies** — 8,788 thread replies, each linked to its parent message
- **reactions** — 4,842 emoji reactions, each linked to its message

Why split into 3? If you want to count reactions, you query only the reactions collection — you do not load all messages. In a real-time system, new reactions can be inserted without touching the original message at all.

What is an Index?

An index in a database is like the index at the back of a book. Instead of reading every page to find a term, you go to the index and it tells you exactly which pages to look at. Without an index, finding all messages from "all-week1" means scanning all 10,814 documents. With an index on channel, MongoDB jumps directly to the right documents.

Should you index everything? No. Every index has two costs: storage space and slower writes (every insert must update all indexes). Index only fields you search or sort by frequently. We indexed: channel, user, ts, and parent_message_id.

PostgreSQL Schema Design — 3 Tables

Table	Rows	What it stores
message_features	19,602	Per-message ML features: sentiment, type, hour, day
user_features	1,205	Per-user stats: message count, reply count, dominant t
daily_sentiment	104	Per-day sentiment score and message count

Key SQL data types used:

- SERIAL PRIMARY KEY — auto-generates unique ID for every row
- TEXT — stores any length of text
- FLOAT — stores decimal numbers (e.g. 0.342 for sentiment)
- INTEGER — stores whole numbers
- NOT NULL — this column must always have a value
- TIMESTAMP DEFAULT NOW() — auto-records the exact insert time

Installing Python Drivers

```
# Install Python library to talk to MongoDB pip install pymongo # Install Python library to talk to PostgreSQL pip install psycopg2-binary
```

7. Task 4 — Streamlit Dashboard

What is Streamlit?

Streamlit is a Python library that turns your Python code into an interactive web app — with charts, tables, dropdowns, sliders — without needing to know HTML, CSS, or JavaScript. You write Python, it gives you a website.

```
pip install streamlit # Run the dashboard streamlit run app/dashboard.py # Opens automatically at http://localhost:8501
```

Dashboard Sections

- **Sidebar** — Channel filter dropdown, live stats (messages, channels, users)
- **Key Metrics** — 4 boxes: total messages, channels, unique users, total reactions
- **Top & Bottom Users** — Tabs for messages, replies, reactions, mentions
- **Channel Activity** — Scatter plot with top 5 channels labelled
- **Message Classification** — Bar chart + pie chart side by side
- **ML Analysis Results** — Time diff histograms, topic model, sentiment charts
- **Raw Data Explorer** — Scrollable table with a row-count slider

> ■ The "Filter by channel" dropdown in the sidebar is interactive. Selecting a specific channel updates the entire dashboard to show only that channel's data. User IDs appear as U03V1AM5TF... because the data is anonymized.

8. Task 5 — Docker, AWS & Terraform

Docker

Docker packages your entire application — the code, Python version, all packages, all settings — into one single box called a container. That container runs identically on any computer in the world. This solves the "works on my machine" problem permanently.

Real life analogy: Before shipping containers existed, loading a cargo ship was chaos — boxes of different shapes, fragile items mixed with heavy ones. After shipping containers, everything goes into a standard metal box. The ship doesn't care what's inside. Docker does exactly that for software.

Concept	What it means
Image	Blueprint of your app — read-only snapshot of code + packages
Container	A running instance of an image (like a meal from a recipe)
Dockerfile	Instructions file for building your image
Docker Hub	Website to store and share images (like GitHub for Docker)

```
# Example Dockerfile FROM python:3.11 COPY . /app WORKDIR /app RUN pip install -r requirements.txt CMD ["streamlit", "run", "app/dashboard.py"]
```

Docker workflow:

```
docker build -t gebeya-dashboard . # build image from Dockerfile docker run -p 8501:8501 gebeya-dashboard # run a container docker push btsm/gebeya-dashboard # upload to Docker Hub
```

AWS (Amazon Web Services)

AWS is Amazon's cloud computing platform. Instead of buying a physical server, you rent computing power from Amazon's data centres and pay only for what you use.

Real life analogy: Buying your own server = buying a generator. Using AWS = using the electricity grid. You pay only for what you use, someone else maintains the infrastructure.

AWS Service	What it does	Our use case
EC2	Virtual servers (rent a computer by the hour)	Run the Streamlit dashboard
S3	File storage in the cloud	Store the anonymized Slack data privately
RDS	Managed PostgreSQL/MySQL	Host PostgreSQL in the cloud
Lambda	Serverless functions (no server to manage)	Run small Python jobs on trigger
ECR	Private Docker image registry	Store our Docker images on AWS

Why AWS matters: Currently the Streamlit dashboard only runs on your Mac. If you close the terminal, it stops. Deploying to AWS means it runs 24/7 on Amazon's servers, anyone with the URL can access it,

and it scales automatically with traffic.

Terraform

Terraform lets you create and manage cloud infrastructure by writing code instead of clicking through websites. This is called Infrastructure as Code (IaC).

Real life analogy: Think of building an IKEA bookshelf. Without Terraform = someone describes it to you verbally, you build from memory. With Terraform = you have the official IKEA instruction manual. Anyone can follow it and build the exact same shelf anywhere.

```
# Example Terraform script (hello world for AWS) provider "aws" { region = "us-east-1" } resource "aws_instance" "web_server" { ami = "ami-0c55b159cbfafe1f0" instance_type = "t2.micro" tags = { Name = "Gebeya-Dashboard-Server" } }
```

```
terraform init # download required plugins terraform plan # preview what will be created (nothing happens yet) terraform apply # create the infrastructure on AWS terraform destroy # delete everything
```

How Docker, AWS & Terraform Work Together

```
You write code on your Mac ↓ Docker builds an image of your app ↓ GitHub Actions pushes the image to Docker Hub automatically ↓ Terraform creates an EC2 server on AWS ↓ The EC2 server pulls your Docker image ↓ Your Streamlit dashboard is live on the internet 24/7
```

9. Questions & Answers

Q: What is a Jupyter Notebook?

A Jupyter notebook is an interactive document that mixes code, text, charts, and outputs in one file (.ipynb). You write code in cells and run them one by one. The output (charts, tables, text) appears directly below each cell. It is saved as a JSON file — when you press Ctrl+S, it saves the code AND the outputs (including charts embedded as images).

Q: What is a pandas DataFrame?

A DataFrame is a table — rows and columns, like an Excel spreadsheet but in Python. Each column has a name and a data type. You can filter, sort, group, and compute on it very efficiently. In this project, every message becomes one row in a DataFrame.

Q: What is a Slack Unix timestamp?

Slack stores message times as a Unix timestamp — the number of seconds since January 1st 1970 (called "the epoch"). For example, 1661774400.0 means "1,661,774,400 seconds after January 1st 1970" = August 29th 2022. The parse_timestamp() function in utils.py converts this to a readable datetime.

Q: What are stop words?

Stop words are common English words that carry no useful meaning for analysis: the, is, a, and, to, of, etc. We remove them before topic modelling so the algorithm focuses on meaningful words. The NLTK library provides a pre-built list of English stop words.

Q: What is the difference between supervised and unsupervised learning?

Supervised learning: you have labeled data (e.g. 1000 messages already tagged as Question/Comment/Answer) and train a model to predict labels for new data. Unsupervised learning: no labels exist. The model finds patterns on its own. LDA topic modelling is unsupervised — we did not tell it what the topics should be.

Q: Why do topics look random (hello, guys, please, error)?

LDA works best on long formal documents like news articles or research papers. Slack messages are short, informal, full of emojis and slang. The topics are messier as a result. Also, names that appear frequently (like Stephanie) get treated as topic keywords. This is expected and should be noted in your report.

Q: What is sentiment polarity?

Polarity is a number from -1 to +1 measuring how positive or negative text is. +1 = very positive ("This is amazing, I love it!"), -1 = very negative ("Terrible, never again"), 0 = neutral ("Here is the link"). TextBlob calculates this by looking up each word in a pre-built dictionary where every word already has a polarity score.

Q: What is the difference between MongoDB and PostgreSQL?

MongoDB stores documents (flexible JSON-like objects — each can have different fields). PostgreSQL stores rows in tables (rigid — every row must have the same columns). MongoDB is best for raw unpredictable data. PostgreSQL is best for structured computed data. We used MongoDB for raw Slack messages and PostgreSQL for our computed ML features.

Q: What is an index in a database?

An index is like the index at the back of a book — instead of scanning every page, you jump directly to what you need. Without an index, MongoDB scans all 10,814 messages to find ones from "all-week1". With an index on channel, it jumps directly. You should NOT index everything — every index slows down writes and uses disk space.

Q: What is Streamlit?

Streamlit is a Python library that converts Python scripts into interactive web apps. You write Python code using Streamlit functions (st.title, st.chart, st.selectbox) and it automatically renders as a website. No HTML, CSS, or JavaScript knowledge needed.

Q: What is the difference between Docker image and container?

An image is the blueprint — a read-only snapshot of your app and everything it needs (like a recipe). A container is a running instance of that image (like the actual meal made from the recipe). You can run 10 containers from one image simultaneously.

Q: What is Infrastructure as Code (IaC)?

IaC means treating your cloud infrastructure the same way you treat your application code — writing it in files, version controlling it with Git, and applying it automatically. Instead of clicking through the AWS website to create servers, you describe them in .tf files and run terraform apply. Terraform then creates exactly what you described.

Summary

Task	Status	Key Deliverable
Task 1	Complete	EDA notebook — 6 analysis sections, all questions ans
Task 2	Complete	ML notebook — classification, LDA, sentiment, MLFlow
Task 3	Complete	MongoDB (3 collections) + PostgreSQL (3 tables)
Task 4	Complete	Streamlit dashboard with 7 interactive sections
Task 5	In Progress	Docker + Terraform + GitHub Actions
GitHub Push	Pending	Waiting for mentor guidance: personal vs company rep

End of Document